ThaiLLM

MEDICAL · INSTALL WORKSHOP

Installing ThaiLLM Medical Application

A 30-minute guided tour of installing the four modules that make up the ThaiLLM Medical application.

PRESENTER	DATE	DURATION
Kasidit Phoncharoen	22 Apr 26	30 minutes

PRESENTER

Hi, I'm Kasidit Phoncharoen

Lead Software Developer on the ThaiLLM Medical project. I drive the architecture across the four modules you'll see today — backend, frontend, prescreen-rulesets, and fact-vector-db — and own the end-to-end install path we're about to walk through.

ROLE
Lead Software Developer

PROJECT
ThaiLLM Medical

CONTACT
kasiditp_pro@vistec.ac.th

BUILT AT
VISAI
VISTEC
VIDYASIRIMEDHI
INSTITUTE OF SCIENCE AND TECHNOLOGY

ORIENTATION

What is ThaiLLM Medical

A Thai-language medical assistant application, built as four cooperating modules. Today we install each one, in order, and end with a live smoke test against the running system.

MODULE 01 **backend**

The central API server on `:3001`. Handles auth, chat orchestration, and data persistence — owns Postgres and RabbitMQ. Every other module talks to it.

MODULE 02 **frontend**

The user-facing web app on `:3000`. Next.js 15 + React 19 + Tailwind — renders the chat UI, prescreen flow, and admin console.

MODULE 03 **prescreen-rulesets**

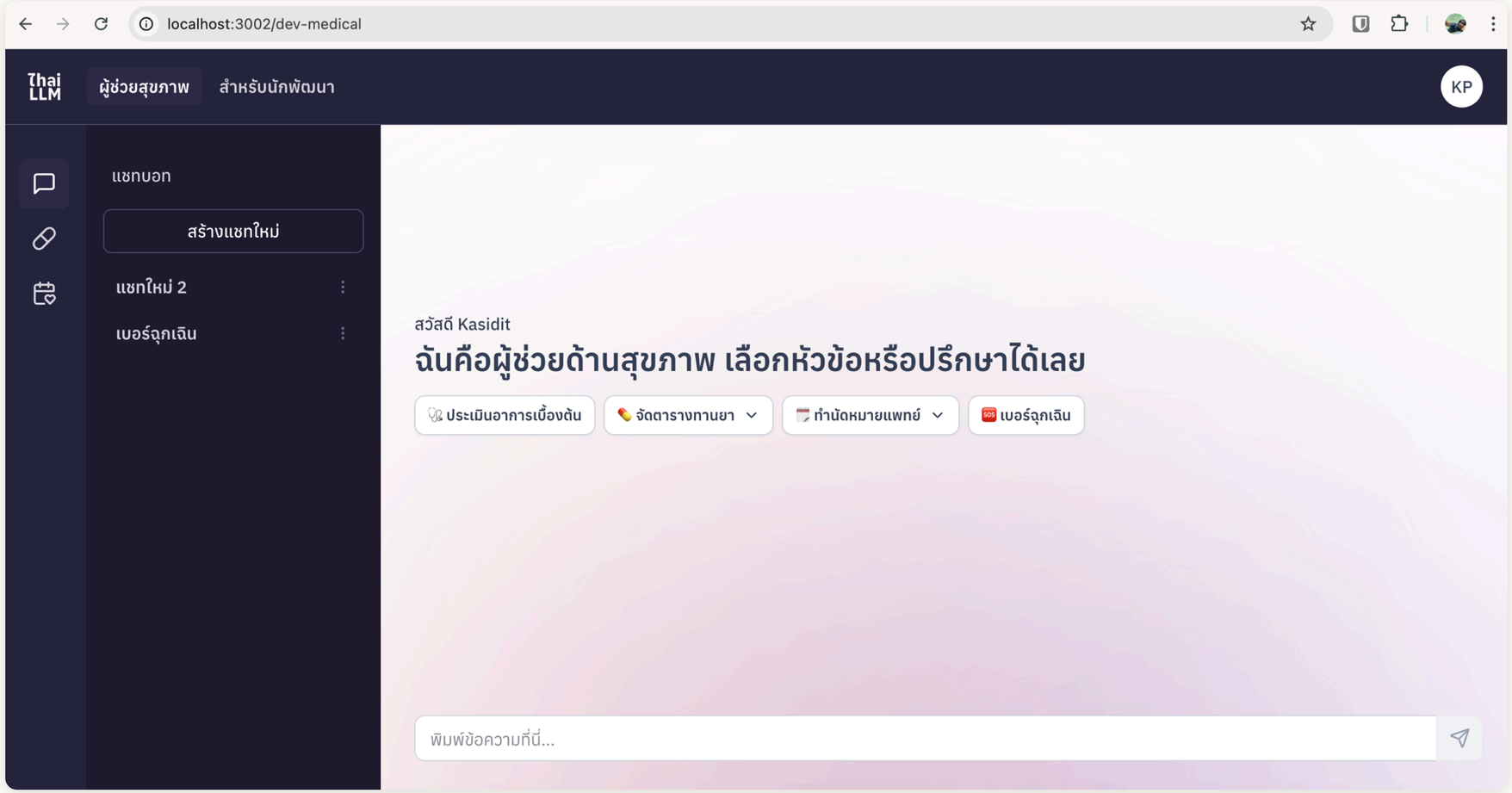
The clinical pre-triage engine on `:8080`. Rule-based YAML (not an LLM) written by doctors — routes patients to department, narrows differential diagnosis, and assigns severity.

MODULE 04 **fact-vector-db**

The medical knowledge retriever on `:41000`. GPU-backed embedding + rerank over a curated Thai medical corpus. Needs an NVIDIA GPU with 11–12 GB VRAM.

QUICK DEMO

A quick look at ThaiLLM Medical



LOCALHOST:3000/DEV-MEDICAL · ผู้ช่วยสุขภาพ

WHAT YOU'LL SEE

01 **Chat** with the assistant in Thai — everyday health questions.

02 **ประเมินอาการเบื้องต้น** — run a symptom prescreen end-to-end.

03 **จัดตารางทานยา** & **ทำนัดหมายแพทย์** — tool calls at work.

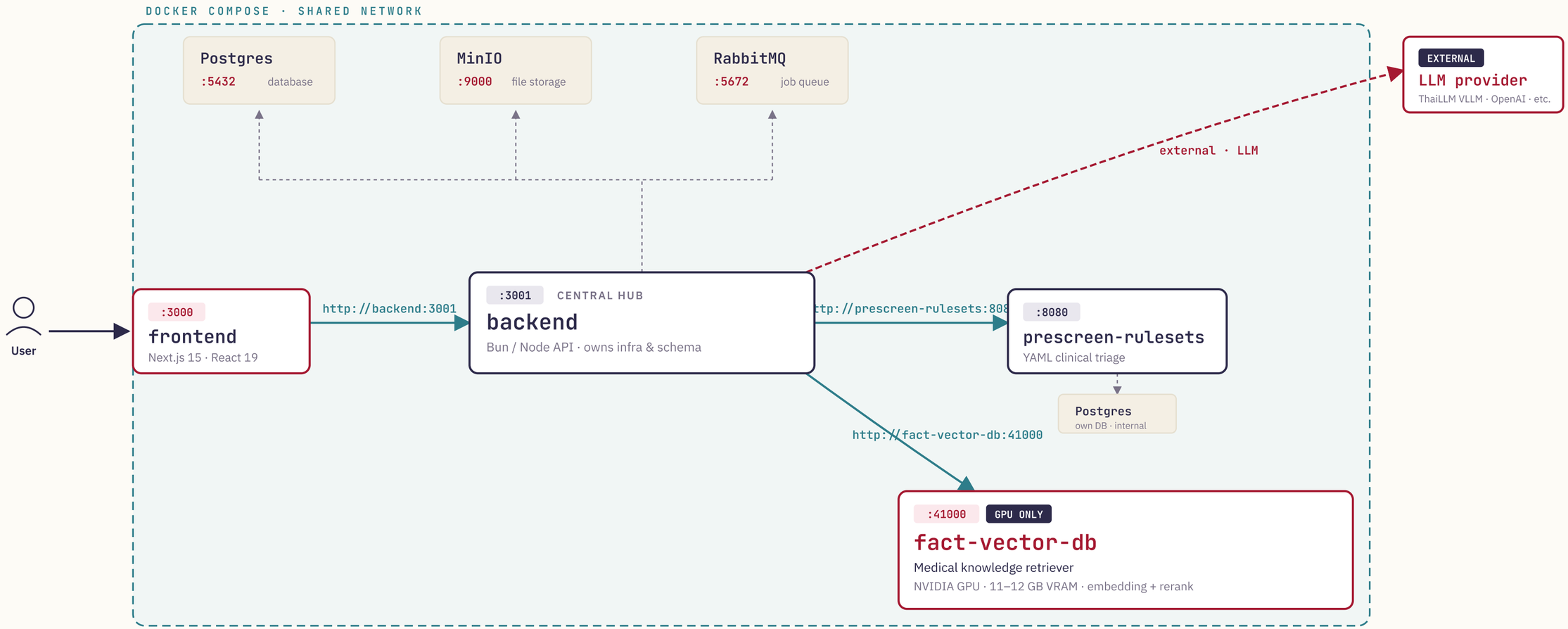
04 **เบอร์ฉุกเฉิน** — red-flag escalation path.

UP NEXT

Then we'll crack it open — architecture, tool calling, and each module in turn.

ARCHITECTURE

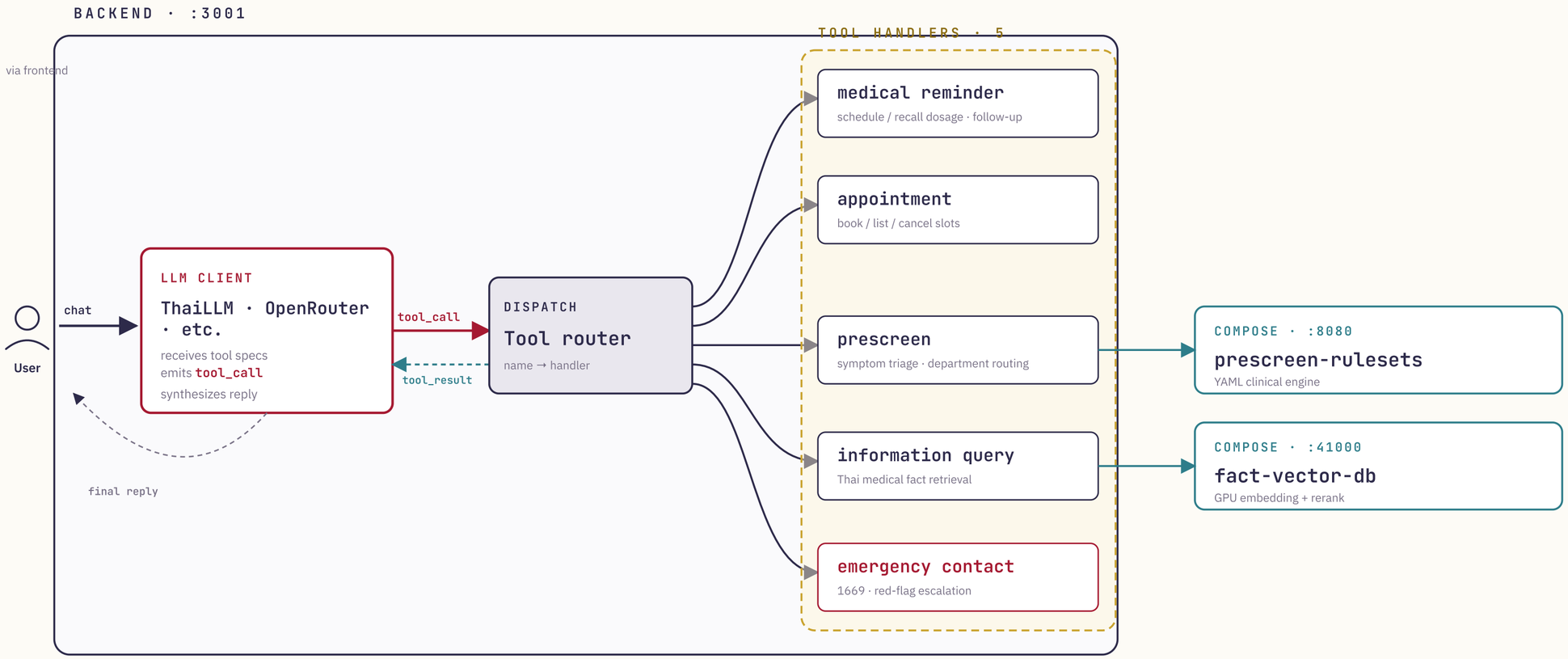
One docker compose stack · shared network



docker compose boundary (shared network) — Intra-compose HTTP (service-name DNS) — External call (crosses compose) --- Dependency (infra → service)

INSIDE THE BACKEND

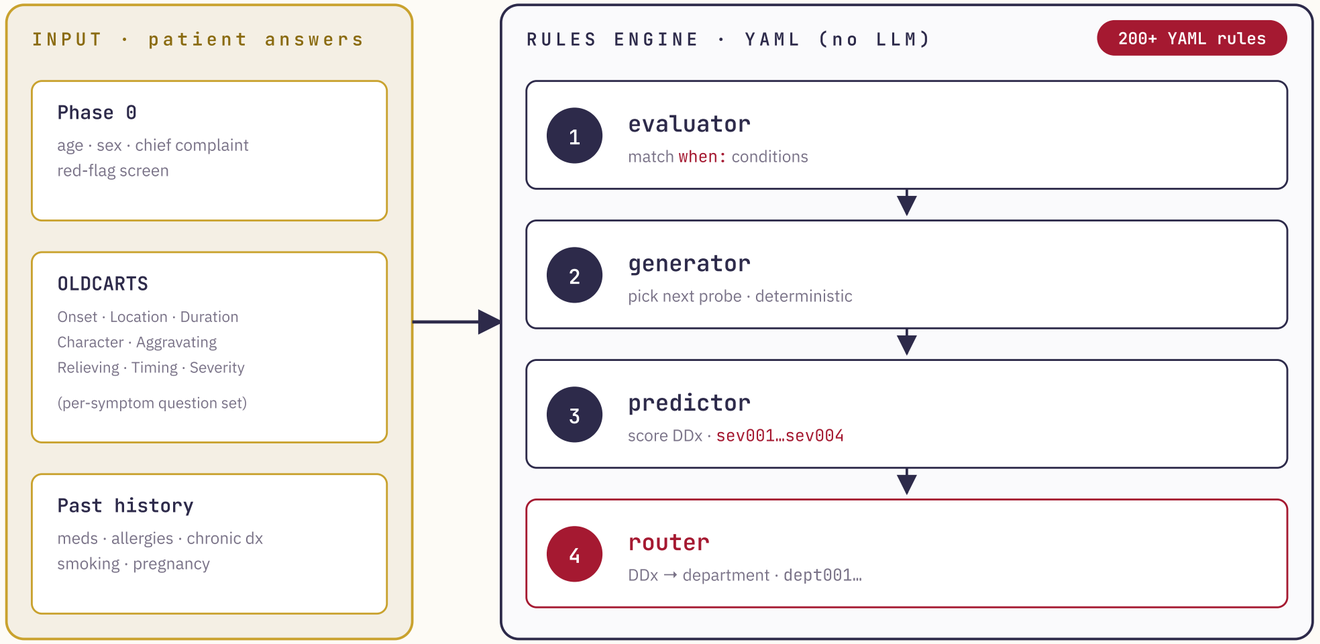
How tool calling works in backend



— LLM emits tool_call --- Tool result back to LLM — Tool hops into compose service — Dispatch & chat

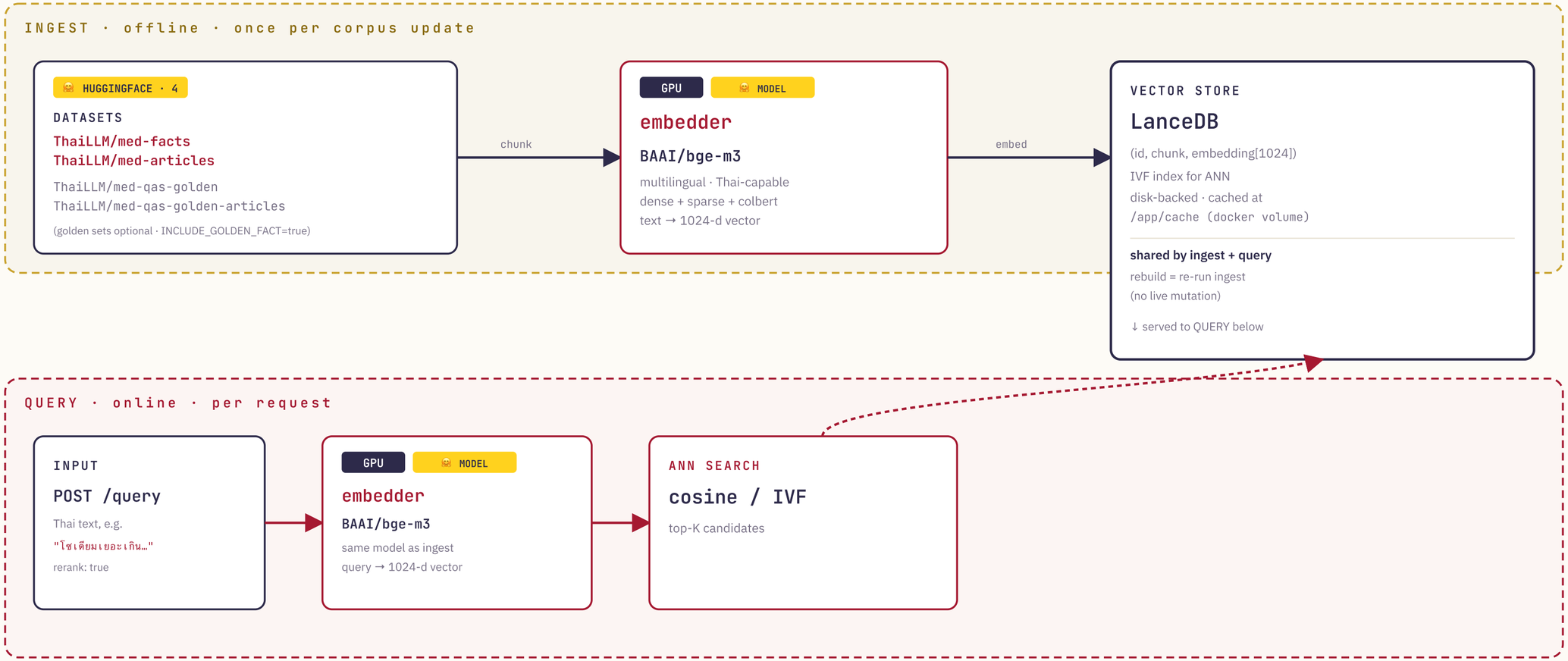
MODULE INTERNALS

prescreen-rulesets — rule-based triage



MODULE INTERNALS

fact-vector-db — embed once, retrieve fast



PART II

Installation Guide

From zero to a running stack — prerequisites, one-shot bootstrap, then each module live.

SETUP

Prerequisites on my machine

You don't need any of this — I am the one installing. This is what's already on my laptop, and what you'd install first if you reproduced this after the workshop.

REQUIRED · WON'T START WITHOUT THESE

1. **Docker Desktop**
   Everything ships as one `docker compose` stack. Docker Desktop bundles engine + compose on macOS / Windows; on Linux, engine + compose plugin is equivalent.

2. **git**
   Clone the orchestrator + four service repos.

3. **LLM API keys**
   `backend` needs `LLM_API_KEY` (ThaiLLM VLLM / OpenAI / etc.); `prescreen-rulesets` will not start without `OPENAI_API_KEY` or `OPENROUTER_API_KEY` (or `SKIP_GENERATOR=true`).

4. **Pre-filled root `.env`**
   One `.env` at the workshop repo root feeds every service via compose. `cp .env.example .env` then edit in real values.

5. **~16 GB free disk**
   Images, build caches, and persisted volumes.

OPTIONAL · ONLY FOR NATIVE OR GPU SETUPS

1. **Node.js `20.15.0`**
   Pinned version — only if you run backend / frontend natively instead of in Docker.

2. **`bun` + `pnpm`**
   Only needed for native backend / frontend installs — Docker images include them.

3. **NVIDIA GPU stack**
   Only for `fact-vector-db`: NVIDIA driver, CUDA-capable GPU with 11–12 GB VRAM, and `nvidia-container-toolkit` for `--gpus`.

START HERE

Clone the workshop & root .env

github.com/visai-ai/thaillm-workshop

SCAN TO OPEN

~ — one-time bootstrap

```
# 1. Clone the workshop orchestrator
$ git clone https://github.com/visai-ai/thaillm-workshop.git
$ cd thaillm-workshop

# 2. Clone the 4 service repos into services/
$ mkdir -p services
$ git clone https://github.com/visai-ai/thaillm-frontend.git            services/frontend
$ git clone https://github.com/visai-ai/thaillm-backend.git             services/backend
$ git clone https://github.com/visai-ai/thaillm-fact-vector-db.git      services/fact-vector-db
$ git clone https://github.com/visai-ai/thaillm-prescreen-rulesets.git services/prescreen-rulesets

# 3. Create root .env — we'll walk the values in each demo
$ cp .env.example .env
```

01 DEMO · BACKEND

Module 1 — backend commands

~ / backend — docker

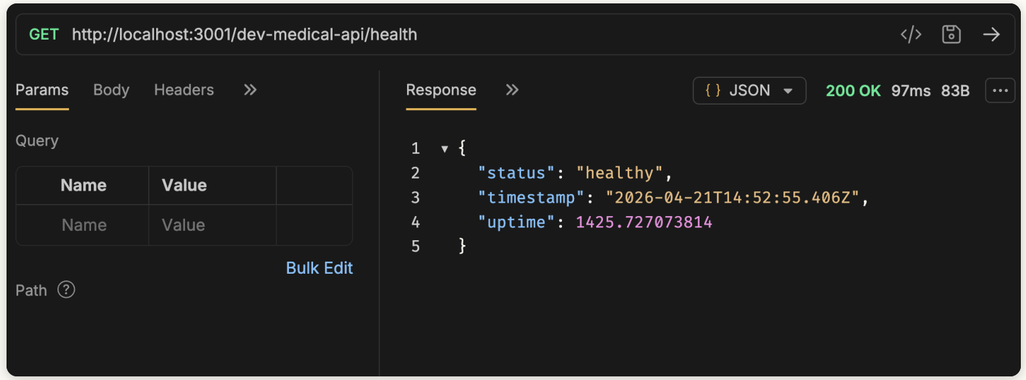
```
# Bootstrap already done (clone + .env). Now build & run.

# 1. Build all service images
$ docker compose build

# 2. Bring the whole stack up (detached)
$ docker compose up -d

# 3. Watch backend come up
$ docker compose logs -f backend

# 4. Status of every service
$ docker compose ps
```

VERIFY · OPEN

`http://localhost:3001/dev-medical-api/health`

KEY ENV TO PRE-FILL

`DATABASE_URL`, `BETTER_AUTH_SECRET`, `ARENA_ENCRYPTION_KEY` (generate with `openssl rand -hex 32`)



GET /DEV-MEDICAL-API/HEALTH · 200 OK

01 DEMO · BACKEND

Module 1 — what just happened

01 `backend` **is the application's API server** — everything else talks to it.

02 `docker compose up -d` **brought up the whole stack**: the four service containers (backend, frontend, prescreen-rulesets, fact-vector-db) plus supporting infra (Postgres, MinIO, RabbitMQ) on one shared compose network.

03 **Migrations run on startup** — backend boots, drizzle creates tables, then the API listens on `:3001`.

04 **Working?** `docker compose ps` shows every service `Up`; `curl http://localhost:3001/dev-medical-api/health` returns `{"status":"ok"}`.

GOTCHA

Inside the compose network, services reach each other by service name (e.g. postgres, backend) — **not** localhost. From your host machine you still hit localhost:<port> because ports are published.

02 DEMO · FRONTEND

Module 2 — frontend commands

~ / frontend — docker

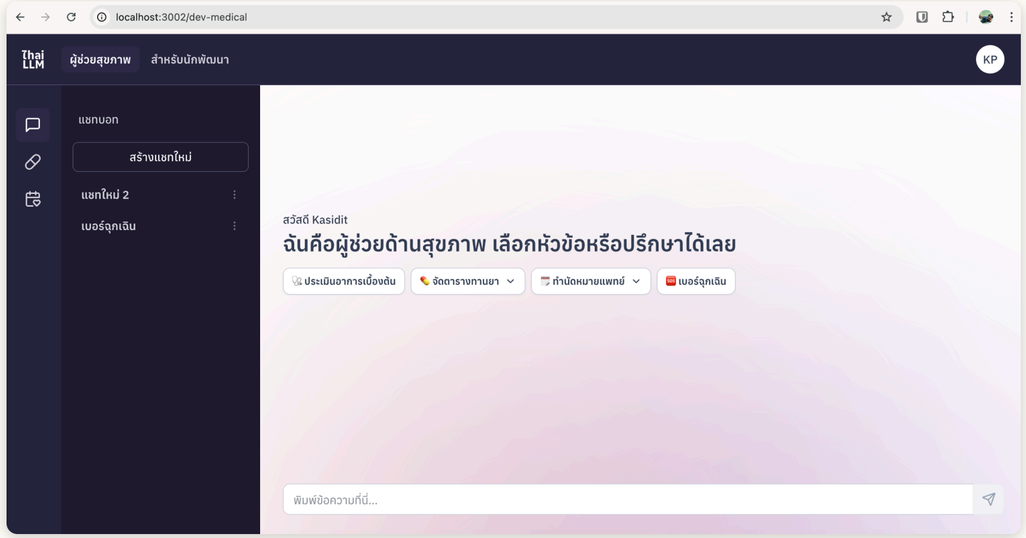
```
# Already up from the master compose. Inspect / iterate:

# 1. Check status
$ docker compose ps frontend

# 2. Follow logs
$ docker compose logs -f frontend

# 3. After changing .env, rebuild just this service
# (NEXT_PUBLIC_* is inlined at build time)
$ docker compose up -d --build frontend

# 4. Restart without rebuilding
$ docker compose restart frontend
```

VERIFY · OPEN

`http://localhost:3000/dev-medical`

BUILD-TIME INLINING

Next.js bakes `NEXT_PUBLIC_*` at build time. Change these in the root `.env` and then `docker compose up -d --build frontend` to pick them up.



ผู้ช่วยสุขภาพ · LANDING SCREEN

02 DEMO · FRONTEND

Module 2 — what just happened

01 `frontend` **is the Next.js 15 + React 19 web app** — the UI users actually see.

02 It talks to `backend` over HTTP at `http://localhost:3001`. If the backend is not running, most pages will show an error state.

03 **Tailwind CSS 4 + shadcn/ui** components provide the styling system.

04 **Working?** Home page renders in the browser; DevTools Network tab shows successful calls to `:3001`.

> **GOTCHA**
> Next.js inlines NEXT_PUBLIC_* vars **at build time** — .env.local must exist *before* docker build. Also, start backend first so the frontend has something to call.

03 DEMO · PRESCREEN-RULESETS

Module 3 — prescreen-rulesets commands

~ / prescreen-rulesets — docker compose

```
# Already up from the master compose. Inspect / iterate:

# 1. Check status + follow logs
$ docker compose ps prescreen-rulesets
$ docker compose logs -f prescreen-rulesets

# 2. Restart after editing .env
$ docker compose restart prescreen-rulesets

# 3. Hit the API directly from your host
$ curl http://localhost:8080/health
```

VERIFY · OPEN

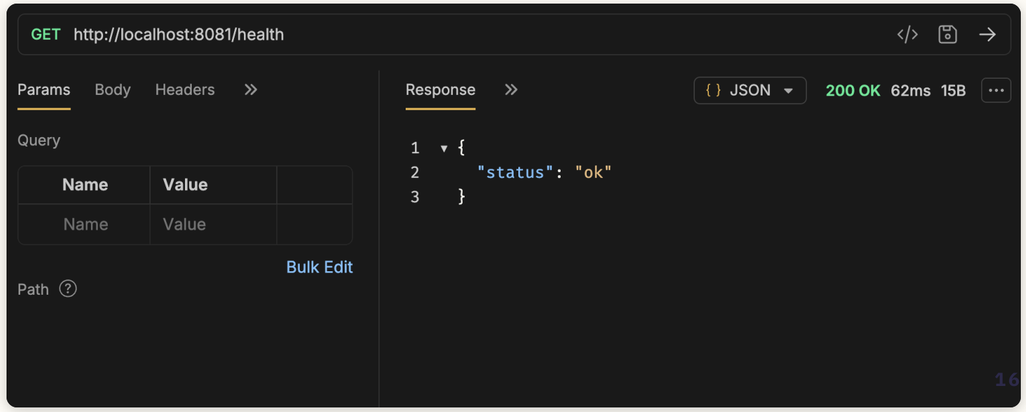
`http://localhost:8080`

OWN POSTGRES

prescreen-rulesets ships its own Postgres inside the compose stack — reachable internally by service name, and doesn't fight with backend's Postgres.

REQUIRED ENV

Server won't start without `OPENAI_API_KEY` or `OPENROUTER_API_KEY` — or set `SKIP_GENERATOR=true` in `.env`.



03 DEMO · PRESCREEN-RULESETS

Module 3 — what just happened

01 `prescreen-rulesets` **is the clinical pre-triage engine.** It routes patients to the right department, narrows the differential diagnosis, and assigns severity.

02 It is **rule-based YAML, not an LLM**. The rules were written by doctors.

03 Brought up as part of the master `docker compose up`, along with its own Postgres — both live inside the compose network.

04 **Working?** `curl http://localhost:8080/health` returns `{"status":"ok"}`.

GOTCHA

Needs OPENAI_API_KEY or OPENROUTER_API_KEY in the root .env (or SKIP_GENERATOR=true) — the server exits on boot without one of them.

04 DEMO · FACT-VECTOR-DB

Module 4 — fact-vector-db commands

! **GPU recommended.** NVIDIA GPU with **11–12 GB VRAM** + ~10 GB CUDA/PyTorch on first run. **CPU works too** — expect **hours of startup**.

~ / fact-vector-db — docker (GPU host)

```
# Already up from the master compose. GPU + shm-size +
# cache volume are wired in the workshop docker-compose.yaml.

# 1. Check status + follow logs
$ docker compose ps fact-vector-db
$ docker compose logs -f fact-vector-db

# 2. Verify (Thai sodium query)
$ curl -X POST http://localhost:41000/query \
    -H "Content-Type: application/json" \
    -d '{"query":"โซเดียมเยอะเกินส่งผลให้เกิดอะไรได้บ้าง","rerank":true}'
```

VERIFY · ON GPU HOST

```
$ curl -X POST http://localhost:41000/query \
    -H "Content-Type: application/json" \
    -d '{"query":"โซเดียมเยอะเกินส่งผลให้เกิดอะไรได้บ้าง","rerank":true}'
```

QUERY, TRANSLATED

"What can happen if I have too much sodium?" Retrieval returns top-matching medical fact snippets.

04 DEMO · FACT-VECTOR-DB

Module 4 — what just happened

01 `fact-vector-db` **is the medical fact retriever** — the knowledge-lookup layer the MCP server calls.

02 It is **GPU-backed** because the embedding and rerank models are heavy. In production it runs on an NVIDIA A100 40 GB.

03 The Thai verify query asks *"What can happen if I have too much sodium?"* — retrieval returns the top-matching medical fact snippets.

04 **Working?** (on a GPU host) the Thai sodium `curl` returns JSON with `results` containing ranked fact snippets.

WHERE IT RUNS

Not on a workshop laptop — install on a dedicated GPU host and point the backend at it.

VALIDATION

End-to-end smoke test

Everything we just installed, wired together. If the final screen appears, the system is correctly plumbed.

CURRENTLY RUNNING

:3001 backend • up · 4 infra containers	:3000 frontend • up	:8080 prescreen-rulesets • up	:41000 fact-vector-db • up

WALK-THROUGH

STEP 01	STEP 02	STEP 03	STEP 04	STEP 05	STEP 06 · RESULT
Open localhost:3000 in the browser	Start a new prescreen session	Answer Phase 0 demographics	Pick a primary symptom (chest pain)	Watch triage: OLDCARTS → history → OPD	Department · DDx · Severity

WRAP-UP

What we did today · what you now have

WHAT WE DID

01 Walked the **architecture** — one `docker compose` stack, four services sharing a network, LLM provider external.

02 Looked inside **tool calling**, **prescreen-rulesets**, and **fact-vector-db** — how the backend routes tool calls and retrieves medical facts.

03 Bootstrapped from the **workshop repo** — clone the orchestrator + 4 service repos, one root `.env`, `docker compose up -d`.

04 Verified each module — backend, frontend, prescreen, fact-vector-db — and ran the end-to-end smoke test.

WHAT YOU NOW HAVE

FULL STACK · LOCAL
A running ThaiLLM Medical instance on your machine, reachable at `localhost:3000`.

CLEAR MENTAL MODEL
You know which service does what, and how they talk to each other over the compose network.

REPRODUCIBLE SETUP
The orchestrator repo + root `.env` — anyone on your team can stand the same stack up from scratch.

A STARTING POINT
Hack on rules, swap the LLM provider, add a tool handler — the code is yours to extend.

APPENDIX

Links used in this workshop

GITHUB · WORKSHOP REPOS

github.com/visai-ai/thaillm-workshop · orchestrator
github.com/visai-ai/thaillm-backend
github.com/visai-ai/thaillm-frontend
github.com/visai-ai/thaillm-prescreen-rulesets
github.com/visai-ai/thaillm-fact-vector-db

🤗 HUGGINGFACE · DATASETS

huggingface.co/ThaiLLM · org
huggingface.co/datasets/ThaiLLM/med-facts
huggingface.co/datasets/ThaiLLM/med-articles
huggingface.co/datasets/ThaiLLM/med-qas-golden
huggingface.co/datasets/ThaiLLM/med-qas-golden-articles

🤗 HUGGINGFACE · MODELS

huggingface.co/BAAI/bge-m3 · embedder
huggingface.co/BAAI/bge-reranker-v2-m3 · reranker

LLM PROVIDERS

thaillm.or.th · VLLM gateway
openrouter.ai
platform.openai.com

TOOLING

docker.com/products/docker-desktop
git-scm.com/downloads

Questions?

If you have any questions, reach out:

Facebook · facebook.com/ISTVISTEC

Email · kasiditp_pro@vistec.ac.th

THAILLM MEDICAL · INSTALL WORKSHOP

Thank you

Kasidit Phoncharoen · Lead Software Developer